CineVerse — System Invariants

Purpose

System invariants are rules that must always remain true regardless of concurrency, retries, duplicate messages, service failures, or infrastructure failures.

These invariants define the correctness guarantees of CineVerse.

Any architectural or implementation decision that violates these invariants must be rejected or redesigned.

---

1. Seat Booking Invariants

INV-001 — No Double Booking

A seat for a specific show must never belong to more than one successful booking.

```
For a given:

(show_id, seat_id)

At most one active/confirmed booking may own the seat.
```

Expected result:

```
10,000 concurrent requests
        ↓
Same show
        ↓
Same seat
        ↓
Exactly 1 successful reservation
        ↓
0 double bookings
```

INV-002 — A Seat Has One Authoritative State

At any point in time, a seat must have exactly one authoritative state.

Possible states:

```
AVAILABLE
HELD
PAYMENT_PENDING
CONFIRMED
EXPIRED
```

The system must not allow ambiguous or conflicting seat states.

INV-003 — Only Valid State Transitions Are Allowed

Valid transitions must be explicitly defined.

Example:

```
AVAILABLE
    ↓
HELD
    ↓
PAYMENT_PENDING
    ↓
CONFIRMED
```

Expiry:

```
HELD
    ↓
EXPIRED
    ↓
AVAILABLE
```

Invalid transitions must be rejected.

Examples:

```
CONFIRMED → HELD        ❌
CONFIRMED → AVAILABLE   ❌
EXPIRED → CONFIRMED     ❌
```

INV-004 — Only One User Can Hold a Seat

At any given time, a seat can have at most one active hold.

Concurrent hold requests must result in:

```
One successful hold
+
All other requests rejected
```

INV-005 — Expired Holds Cannot Be Confirmed

Once a seat hold expires, the original booking request must not be able to confirm that seat.

```
HELD
  ↓
EXPIRED
  ↓
AVAILABLE
```

The old booking attempt must not be able to transition:

```
EXPIRED → CONFIRMED
```

---

2. Booking Invariants

INV-006 — A Confirmed Booking Must Have Confirmed Seats

A booking cannot reach:

```
BOOKING_CONFIRMED
```

unless all required seats have been successfully confirmed.

INV-007 — A Confirmed Booking Must Have Successful Payment

A booking cannot be considered successfully confirmed unless the associated payment has reached a successful state.

```
Payment Success
      ↓
Booking Confirmation
```

A failed or pending payment cannot produce a confirmed booking.

INV-008 — Booking State Must Be Durable

The current state of a distributed booking workflow must not exist only in application memory.

If the Saga orchestrator or service crashes:

```
Service Crash
    ↓
Restart
    ↓
Load Durable State
    ↓
Resume or Compensate
```

The system must not lose the workflow state because of a process restart.

INV-009 — Booking State Transitions Must Be Valid

A booking must follow valid lifecycle transitions.

Example:

```
CREATED
   ↓
SEAT_HELD
   ↓
PAYMENT_PENDING
   ↓
PAYMENT_CONFIRMED
   ↓
BOOKING_CONFIRMING
   ↓
BOOKING_CONFIRMED
   ↓
COMPLETED
```

Failure paths may transition into:

```
FAILED
COMPENSATING
REFUND_PENDING
```

Invalid transitions must be rejected.

---

3. Payment Invariants

INV-010 — Payment Must Be Idempotent

The same logical payment request must not result in multiple successful payments.

Example:

```
Same booking
Same idempotency key
10 requests
        ↓
One logical payment
```

Duplicate requests must return or reference the existing payment result.

INV-011 — A Payment Cannot Be Confirmed Twice

A payment must not transition from:

```
SUCCESS
```

to another successful payment operation.

Duplicate payment confirmation messages must have no additional business effect.

INV-012 — Compensation Must Be Idempotent

If a refund or compensation operation is retried multiple times, it must not produce multiple refunds for the same payment.

```
Refund Request
     ↓
Retry
     ↓
Retry
     ↓
One Refund
```

---

4. Saga Invariants

INV-013 — Every Saga Has One Current State

A booking Saga must have one authoritative current state.

Concurrent or duplicate Saga commands must not cause conflicting state transitions.

INV-014 — Compensation Must Be Safe to Retry

Every compensating action must be idempotent.

Examples:

```
Release Seat
Refund Payment
Cancel Booking
```

Retrying compensation must not create additional unwanted side effects.

INV-015 — Payment Success + Booking Failure Must Trigger Compensation

If payment succeeds but booking confirmation fails, the Saga must enter a compensating workflow.

Expected flow:

```
Payment SUCCESS
      ↓
Booking Confirmation FAILED
      ↓
Refund
      ↓
Release Seat
```

The system must not permanently leave the user with a successful payment and no valid booking without entering a recoverable compensation state.

INV-016 — Compensation Failure Must Be Recoverable

If compensation itself fails:

```
Refund
   ↓
FAIL
```

The Saga must enter a durable retryable state such as:

```
REFUND_PENDING
```

The system must not silently lose the compensation operation.

---

5. Event Invariants

INV-017 — Committed Business Data Must Have a Durable Event Source

If a business operation requires an event to be published, the system must not rely on an unsafe dual-write:

```
Database Commit
     ↓
Kafka Publish
```

Instead:

```
Database Transaction
     ├── Business Data
     └── Outbox Event
```

The event must have a durable source before it is considered ready for publication.

INV-018 — Events Must Not Be Lost Due to Temporary Kafka Failure

If Kafka is temporarily unavailable:

```
Kafka DOWN
    ↓
Business Transaction
```

```
    ↓
Outbox
```

The event must remain available for later publication.

After Kafka recovers:

```
Outbox
   ↓
Publisher
   ↓
Kafka
```

INV-019 — Consumers Must Be Idempotent

A consumer receiving the same event multiple times must produce only one business effect.

```
Event 123
   ↓
Process

Event 123
   ↓
Ignore

Event 123
   ↓
Ignore
```

The final business state must be equivalent to processing the event once.

INV-020 — Event Ordering Must Be Explicit

Where ordering is required, events must use an appropriate Kafka partitioning strategy.

The system must explicitly define the ordering key for each event domain.

Examples:

```
Booking lifecycle
→ booking_id
```

```
Seat lifecycle
→ show_id + seat_id
```

Ordering guarantees must not be assumed without defining the partitioning strategy.

---

6. Distributed System Invariants

INV-021 — PostgreSQL Is the Source of Truth for Seat Correctness

Redis must not be the ultimate authority for whether a seat is successfully booked.

Redis may be used for:

- Locking
- Fast-fail behaviour
- Contention reduction
- Performance optimisation

The final correctness decision must be backed by the authoritative persistent data store.

INV-022 — Redis Failure Must Not Cause Double Booking

If Redis becomes unavailable:

```
Redis DOWN
    ↓
Fallback / Alternative Correctness Mechanism
    ↓
Lower Performance
```

The system may become slower, but correctness must be preserved.

INV-023 — Retries Must Not Create Duplicate Business Effects

A network timeout may cause a client or service to retry a request.

Retries must not result in:

```
Duplicate Payment
Duplicate Booking
Duplicate Seat Hold
Duplicate Refund
```

All retryable business operations must have appropriate idempotency protection.

INV-024 — Service Restart Must Not Lose Durable Business State

Restarting any service must not lose:

- Confirmed bookings
- Payment state
- Seat state
- Saga state
- Outbox events
- Idempotency records

INV-025 — Horizontal Scaling Must Preserve Correctness

Adding more application instances must not change the correctness guarantees.

For example:

```
1 Booking Instance
    ↓
Zero double booking

5 Booking Instances
    ↓
Zero double booking

20 Booking Instances
    ↓
Zero double booking
```

Correctness must not depend on in-memory locks inside a single application instance.

---

7. Failure Invariants

INV-026 — Kafka Failure Must Not Corrupt Committed Business State

Kafka being unavailable must not cause already committed database transactions to become inconsistent.

The Outbox pattern must isolate the business transaction from Kafka availability.

INV-027 — Payment Failure Must Not Leave Seats Permanently Held

If payment fails:

```
Payment FAILED
    ↓
Seat Released
```

or the seat must enter a recoverable state that eventually leads to release.

INV-028 — Service Failure Must Lead to Recovery or Explicit Failure

A service crash must not silently leave the system in an unknown state.

After failure, the system must either:

```
Recover
```

or:

```
Enter an explicit retryable/failed state
```

INV-029 — No Silent Failure

Every important failure must be:

- Logged
- Observable through metrics
- Traceable where applicable
- Represented in business state if necessary

A failed operation must not disappear silently.

---

8. Observability Invariants

INV-030 — Every Booking Must Be Traceable

A booking request must have a correlation mechanism that allows engineers to trace its lifecycle across services.

At minimum, the system should be able to correlate:

```
Gateway
   ↓
Booking
   ↓
Inventory
   ↓
Payment
   ↓
Kafka
   ↓
Notification
```

INV-031 — Critical Business Events Must Be Observable

The system should expose metrics for important business operations.

Examples:

```
Successful bookings
Failed bookings
Seat lock conflicts
Payment failures
Saga compensations
Outbox backlog
Kafka consumer lag
```

---

9. Performance Invariants

INV-032 — Performance Must Be Measured, Not Assumed

The system must not claim scalability without benchmark evidence.

Performance tests should measure:

```
Requests per second
P50 latency
P95 latency
P99 latency
Error rate
Resource utilisation
```

INV-033 — Correctness Takes Priority Over Performance

Performance optimisations must never violate correctness.

For example:

```
Faster booking
+
Double booking
=
FAILURE
```

The priority is:

```
Correctness
    ↓
Consistency
    ↓
Reliability
    ↓
Performance
```

Performance improvements must preserve the first three.

---

10. Core System Principle

All architectural decisions in CineVerse must preserve the following fundamental principle:

> **A user should never be charged for a booking that the system cannot correctly account for, and a seat should never be successfully assigned to more than one confirmed booking.**

The ultimate correctness guarantees are:

```
1. No double booking.

2. No duplicate successful payment.

3. No lost committed business event.

4. No unhandled distributed workflow.

5. No silent failure.

6. No correctness regression when scaling horizontally.

7. No correctness regression when infrastructure components temporarily fail.
```

These invariants define the correctness boundary of CineVerse.

Any future feature, optimisation, or architectural change must preserve these invariants.